

430.211 Programming Methodology (프로그래밍 방법론)

Taesup Moon

Department of Electrical and Computer Engineering
Seoul National University

Outline

- Chapter 6: Structures and Classes
 - Structures
 - Classes

Outline

- Chapter 6: Structures and Classes
 - Structures
 - Classes

Ch 6. Structures and Classes

Learning Objectives

- Structures
 - Structure types
 - Structures as function arguments
 - Initializing structures
- Classes
 - Defining, member functions
 - Public and private members
 - Encapsulation, principles of OOP
 - Accessor and mutator functions
 - Structures vs. classes

Structures

- 2nd aggregate data type: struct
- Recall: aggregate meaning "grouping"
 - Recall array: collection of values of same type
 - Structure: collection of values of **different** types
- Treated as a single item, like arrays
- Major difference: **Must first "define"** `struct`
 - Prior to declaring any variables

Structure Types

- Define `struct` globally (typically)
 - a "placeholder" for what our `struct` will "look like"

- Definition:

```
struct CDAccountV1    ← Name of new struct "type"
{
    double balance;    ←
    double interestRate; ← member names
    int term;          ←
};
```

Declare Structure Variable

- With the new structure **type** defined, now we can declare variables of this new type:

```
CDAccountV1  account;
```

- Just like declaring simple types
- Variable `account` is now of type `CDAccountV1`
 - A struct value contains a collection of smaller values called "member values"

Accessing Structure Members

- Dot Operator (.) to access members

```
account.balance  
account.interestRate  
account.term
```

- Called "member variables"
 - The "parts" of the structure variable
 - Different structs can have same name member variables
 - No conflicts

```
struct FertilizerStock  
{  
    double quantity;  
    double nitrogenContent;  
};
```

```
struct CropYield  
{  
    int quantity;  
    double size;  
};
```

```
FertilizerStock superGrow;  
CropYield apples;
```

```
superGrow.quantity  
apples.quantity  
→ No ambiguity!
```

Structure Assignments

- Given structure named `CropYield`

- Declare two structure variables:

```
CropYield apples, oranges;
```

- Both are variables of "struct type `CropYield`"
- Simple assignments are legal:

```
apples = oranges;
```

- Simply copies each member variable of `oranges` into member variables of `apples`

```
apples.quantity = oranges.quantity;  
apples.size = oranges.size;
```

Structures as Function Arguments

- Passed like any simple data type
 - Call-by-value
 - Call-by-reference
 - Or combination
- Can also be returned by a function
 - Return-type is structure type
 - Return statement in function definition sends structure variable back to caller

Structures as Function Arguments : Example

- Ex)

```
CDAccountV1 doubleInterest (CDAccountV1 oldAccount)
{
    CDAccountV1 temp;
    temp = oldAccount;
    temp.interestRate = 2*oldAccount.interestRate;
    return temp;
}
```

- In a main function:

```
CDAccountV1 yourAccount;
yourAccount = doubleInterest (myAccount);
```

Structure Example:

Display 6.1 A Structure Definition (1 of 3)

Display 6.1 A Structure Definition (part 1 of 2)

```
1  //Program to demonstrate the CDAccountV1 structure type.
2  #include <iostream>
3  using namespace std;

4  //Structure for a bank certificate of deposit:
5  struct CDAccountV1
6  {
7      double balance;
8      double interestRate;
9      int term; //months until maturity
10 };

11 void getData(CDAccountV1& theAccount);
12 //Postcondition: theAccount.balance, theAccount.interestRate, and
13 //theAccount.term have been given values that the user entered at the
   //keyboard.
```

An improved version of this structure will be given later in this chapter.

Structure Example:

Display 6.1 A Structure Definition (2 of 3)

```
14  int main( )
15  {
16      CDAccountV1 account;
17      getData(account);

18      double rateFraction, interest;
19      rateFraction = account.interestRate/100.0;
20      interest = account.balance*(rateFraction*(account.term/12.0));
21      account.balance = account.balance + interest;

22      cout.setf(ios::fixed);
23      cout.setf(ios::showpoint);
24      cout.precision(2);
25      cout << "When your CD matures in "
26           << account.term << " months,\n"
27           << "it will have a balance of $"
28           << account.balance << endl;

29      return 0;
30  }
```

Structure Example:

Display 6.1 A Structure Definition (3 of 3)

```
31 //Uses iostream:
32 void getData(CDAccountV1& theAccount)
33 {
34     cout << "Enter account balance: $";
35     cin >> theAccount.balance;
36     cout << "Enter account interest rate: ";
37     cin >> theAccount.interestRate;
38     cout << "Enter the number of months until maturity: ";
39     cin >> theAccount.term;
40 }
```

Sample Dialogue

```
Enter account balance: $100.00
Enter account interest rate: 10.0
Enter the number of months until maturity: 6
When your CD matures in 6 months,
it will have a balance of $105.00
```

Structure Pitfall

- Semicolon after structure definition

- ; **MUST exist:**

```
struct WeatherData
{
    double temperature;
    double windVelocity;
}; ← REQUIRED semicolon!
```

- Required since you "can" declare structure variables in this location

```
struct WeatherData
{
    double temperature;
    double windVelocity;
} dataPoint1, dataPoint2;
```

Hierarchical structures

- Sometimes, we can have structures whose members are themselves smaller structures

```
struct Date
{
    int month;
    int day;
    int year;
};
```

```
struct PersonInfo
{
    double height; //in inches
    int weight; //in pounds
    Date birthday;
};
```

```
PersonInfo person1;
cout << person1.birthday.year;
```

This itself is a structure!
Read from the left to the right!

Hierarchical structures : Example (1/3)

Display 6.2 A Structure with a Structure Member (part 1 of 2)

```
1  //Program to demonstrate the CDAccount structure type.
2  #include <iostream>
3  using namespace std;

4  struct Date
5  {
6      int month;
7      int day;
8      int year;
9  };

10 //Improved structure for a bank certificate of deposit:
11 struct CDAccount
12 {
13     double initialBalance;
14     double interestRate;
15     int term; //months until maturity
16     Date maturity; //date when CD matures
17     double balanceAtMaturity;
18 };

19 void getCDData(CDAccount& theAccount);
20 //Postcondition: theAccount.initialBalance, theAccount.interestRate,
21 //theAccount.term, and theAccount.maturity have been given values
22 //that the user entered at the keyboard.
23
```

This is an improved version of the structure CDAccountV1 defined in Display 6.1.

Hierarchical structures : Example (2/3)

```
24 void getDate(Date& theDate);
25 //Postcondition: theDate.month, theDate.day, and theDate.year
26 //have been given values that the user entered at the keyboard.

27 int main( )
28 {
29     CDAccount account;
30     cout << "Enter account data on the day account was opened:\n";
31     getCDData(account);
32     double rateFraction, interest;
33     rateFraction = account.interestRate / 100.0;
34     interest = account.initialBalance*(rateFraction*(account.term /
35                                         12.0));
36     account.balanceAtMaturity = account.initialBalance + interest;

37     cout.setf(ios::fixed);
38     cout.setf(ios::showpoint);
39     cout.precision(2);
40     cout << "When the CD matured on "
41           << account.maturity.month << "-" << account.maturity.day
42           << "-" << account.maturity.year << endl
43           << "it had a balance of $"
44           << account.balanceAtMaturity << endl;
45     return 0;
```

Hierarchical structures : Example (3/3)

```
45 }
46 //uses iostream:
47 void getCDDData(CDAccount& theAccount)
48 {
49     cout << "Enter account initial balance: $";
50     cin >> theAccount.initialBalance;
51     cout << "Enter account interest rate: ";
52     cin >> theAccount.interestRate;
53     cout << "Enter the number of months until maturity: ";
54     cin >> theAccount.term;
55     cout << "Enter the maturity date:\n";
56     getDate(theAccount.maturity);
57 }

58 //uses iostream:
59 void getDate(Date& theDate)
60 {
61     cout << "Enter month: ";
62     cin >> theDate.month;
63     cout << "Enter day: ";
64     cin >> theDate.day;
65     cout << "Enter year: ";
66     cin >> theDate.year;
67 }
```

Sample Dialogue

```
Enter account data on the day account was opened:
Enter account initial balance: $100.00
Enter account interest rate: 10.0
Enter the number of months until maturity: 6
Enter the maturity date:
Enter month: 2
Enter day: 14
Enter year: 1899
When the CD matured on 2-14-1899
it had a balance of $105.00
```

Initializing Structures

- Can initialize at declaration

- Ex:

```
struct Date
{
    int month;
    int day;
};
Date dueDate = {12, 31};
```

- Declaration provides initial data to all two member variables
 - Error if there are more initializer values than struct members
 - If fewer than struct members, values are used to initialize data members ***in order***
 - ➔ Data member without an initializer is initialized to a zero value of an appropriate type for the variable)

Outline

- Chapter 6: Structures and Classes
 - Structures
 - Classes

Class

- Similar to a structure type
 - But, we can also add member **FUNCTIONS**
- Integral to **object-oriented programming (OOP)**
 - Objects
 - The value of a variable of a class type
 - Contains both **data** and **function** members



Class defines what the object should be.



Each instance of that class is called an **object**.

Class Definitions

- Defined similarly as structures

- Example:

```
class DayOfYear      ← name of a new class type
{
public:               ← On "public" vs. "private" given later
    void output();   ← member function declaration!
    int month;
    int day;
};
```

- Notice that only the member function's **prototype** is included
 - Function's definition is somewhere else

Declaring Objects

- Declared the same as other variables
 - Predefined types, structure types
- Example:
`DayOfYear today, birthday;`
 - Declares two objects of class type `DayOfYear`
- Objects include:
 - **Data**
 - Ex) `month, day`
 - **Operations** (i.e., member functions)
 - Ex) `output()`

Class Member Access

- Dot operator
 - Members are accessed the same as structures!
- Ex)
 - To access the member variables:
`today.month`
`today.day`
 - To access the member function:
`today.output();` ← Invokes the member function

Class Member Functions

- Must define or "implement" class member functions
- Like other function definitions
 - Can be after `main()`
 - Must specify class:

```
void DayOfYear::output()  
{...}
```

 - `::` is called the **scope resolution operator**
(what is the difference with the dot operator?)
 - Instructs compiler "what class" member is from (more than one class may have the member function with the same name!)
 - Item before `::` is called ***type qualifier***

Class Member Functions Definition

- Notice `output()` member function's definition (in the next example)
- When it refers to ***member data*** of the class
 - **No** need to use the dot operator!
- The function will refer to "that object's" data when invoked
 - Example:
`today.output();`
 - Displays "today" object's data

Complete Class Example:

Display 6.3 Class With a Member Function (1 of 3)

Display 6.3 Class with a Member Function (part 1 of 3)

```
1 //Program to demonstrate a very simple example of a class.
2 //A better version of the class DayOfYear will be given in Display 6.4.
3 #include <iostream>
4 using namespace std;

5 class DayOfYear
6 {
7 public:
8     void output( );
9     int month;
10    int day;
11 };

12 int main( )
13 {
14     DayOfYear today, birthday;
15     cout << "Enter today's date:\n";
16     cout << "Enter month as a number: ";
17     cin >> today.month;
18     cout << "Enter the day of the month: ";
19     cin >> today.day;
20     cout << "Enter your birthday:\n";
21     cout << "Enter month as a number: ";
22     cin >> birthday.month;
23     cout << "Enter the day of the month: ";
24     cin >> birthday.day;
```

Normally, member variables are private and not public, as in this example. This is discussed a bit later in this chapter.

Member function declaration

Complete Class Example:

Display 6.3 Class With a Member Function (2 of 3)

```
25     cout << "Today's date is ";
26     today.output( );
27     cout << endl;
28     cout << "Your birthday is ";
29     birthday.output( );
30     cout << endl;

31     if (today.month == birthday.month && today.day == birthday.day)
32         cout << "Happy Birthday!\n";
33     else
34         cout << "Happy Unbirthday!\n";
35     return 0;
36 }
37 //Uses iostream:
38 void DayOfYear::output( )
39 {
40     switch (month)
41     {
42         case 1:
43             cout << "January "; break;
44         case 2:
45             cout << "February "; break;
46         case 3:
47             cout << "March "; break;
```

Calls to the member function output

Member function definition

Complete Class Example:

Display 6.3 Class With a Member Function (3 of 3)

```
49         cout << "April "; break;
50     case 5:
51         cout << "May "; break;
52     case 6:
53         cout << "June "; break;
54     case 7:
55         cout << "July "; break;
56     case 8:
57         cout << "August "; break;
58     case 9:
59         cout << "September "; break;
60     case 10:
61         cout << "October "; break;
62     case 11:
63         cout << "November "; break;
64     case 12:
65         cout << "December "; break;
66     default:
67         cout << "Error in DayOfYear::output.";
68     }
69
70     cout << day;
71 }
```

Sample Dialogue

```
Enter today's date:
Enter month as a number: 10
Enter the day of the month: 15
Enter your birthday:
Enter month as a number: 2
Enter the day of the month: 21
Today's date is October 15
Your birthday is February 21
Happy Unbirthday!
```

A Class's Place

- Class is a full-fledged type!
 - Just like usual data types `int`, `double`, etc.
- Can have variables of a class type
 - We simply call them "`objects`"
- Can have parameters of a class type
 - Call-by-value
 - Call-by-reference
- Can use class type like any other type!

Encapsulation

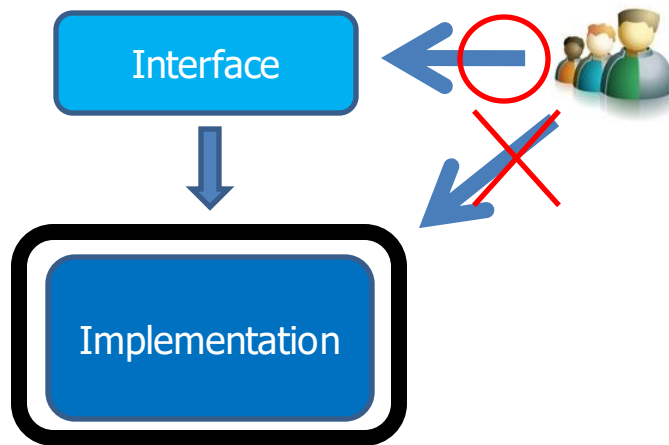
- Not that **any** data type includes
 - Data (range of data)
 - Operations (or functions that can be performed on data)
- Ex)
`int` has:
 - Data: -2147483648 to 2147483647 (for 32 bit int)
 - Operations: +, -, *, /, %, logical, etc.
- Two points
 1. Data type is not just a simple collection of values (operation is needed!)
 2. We do not need to know how the int data type is implemented! (But, we can just use it!)

Abstract Data Types (ADT)

- A data type
 - Collection of data values + set of operations for the values
 - Programmers don't need to know the details on how the values and operations are implemented → “Abstract”!!
 - `int` is an ADT
- Classes
 - ***Programmer-defined*** ADTs
 - WE specify **data**, and the **operations** on our data!
 - Details of how the “operations” are implemented should be hidden from the users → declaration/comments should suffice for using the class
- ***Encapsulation/Information hiding/data abstraction***

Encapsulation – Separation of Interface & Implementation

- How do we realize encapsulation in C++?
 - Data and function access is encouraged to be done only through the **(public) member functions** (interface)
 - Hide the implementation details of the data and functions **(private!)**



- The users of a class need to know only its **interface**.

→ Only need to know **what the class does** rather than **how the class is implemented!**

An important feature of OOP!

Public and Private Members

- In Display 6.3, you need to know the class has two ints
 - Violates encapsulation!
 - No need to know whether a month is stored as a number or a string
- We can utilize public/private members
 - **Public:**
 - “user-accessible”
 - Usually member functions
 - **Private:**
 - Not directly accessible to users
 - Allow manipulation only via member functions!

```
int main( )
{
    DayOfYear today, birthday;
    cout << "Enter today's date:\n";
    cout << "Enter month as a number: ";
    cin >> today.month;
    cout << "Enter the day of the month: ";
    cin >> today.day;
    cout << "Enter your birthday:\n";
    cout << "Enter month as a number: ";
    cin >> birthday.month;
    cout << "Enter the day of the month: ";
    cin >> birthday.day;
```

Public and Private Example

- Modify previous example:

```
class DayOfYear
{
public:
    void input();
    void output();
private:
    int month;
    int day;
};
```

```
DayOfYear today; //This line is OK.
today.month = 12; //ILLEGAL
today.day = 25; //ILLEGAL
cout << today.month; //ILLEGAL
cout << today.day; //ILLEGAL
if (today.month == 1) //ILLEGAL
    cout << "January";
```

- Data is now **private**
 - Must now call public member functions to access
`today.input()`, `today.output()`

Why Encapsulation?

- Example (Box class)

```
class Box {  
public:  
    Box() : height(0), width(0), length(0), area(0), volume(0){}  
  
    void set(double h, double w, double l) {  
        height = h; width = w; length = l;  
        area = 2 * (h*w + w*l + l*h);  
        volume = h*w*l;  
    }  
    void print() {  
        cout << "Box      : (" << height << ", " << width << ", " << length << ")" << endl;  
        cout << "Area      : " << area << endl;  
        cout << "Volume    : " << volume << endl;  
    }  
  
    double height, width, length;  
    double area, volume;  
};  
  
void main() {  
    Box box1;  
    box1.set(1,2,3);  
    box1.print();  
    box1.height = box1.width = box1.length = 30; // ruins data consistency  
    box1.print();  
}
```

Box	:	(1, 2, 3)
Area	:	22
Volume	:	6
Box	:	(30, 30, 30)
Area	:	22
Volume	:	6

Prevent the Danger of Unintended Change

- Example (Box class)

```
class Box {  
public:  
    Box() : height(0), width(0), length(0), area(0), volume(0){}  
  
    void set(double h, double w, double l) {  
        height = h; width = w; length = l;  
        area = 2 * (h*w + w*l + l*h);  
        volume = h*w*l;  
    }  
    void print() {  
        cout << "Box      : (" << height << ", " << width << ", " << length << ")" << endl;  
        cout << "Area      : " << area << endl;  
        cout << "Volume    : " << volume << endl;  
    }  
  
    double height, width, length;  
    double area, volume;  
};  
  
void main() {  
    Box box1;  
    box1.set(1,2,3);  
    box1.print();  
    box1.set(30,30,30); // data are now consistent  
    box1.print();  
}
```

Box	:	(1, 2, 3)
Area	:	22
Volume	:	6
Box	:	(30, 30, 30)
Area	:	5400
Volume	:	27000

Enforcing Encapsulation : Data as Private Members

- Example (Box class)

```
class Box {  
public:  
    Box() : height(0), width(0), length(0), area(0), volume(0){}  
  
    void set(double h, double w, double l) {  
        height = h; width = w; length = l;  
        area = 2 * (h*w + w*l + l*h);  
        volume = h*w*l;  
    }  
    void print() {  
        cout << "Box      : (" << height << ", " << width << ", " << length << ")" << endl;  
        cout << "Area      : " << area << endl;  
        cout << "Volume   : " << volume << endl;  
    }  
private:  
    double height, width, length;  
    double area, volume;  
};  
  
void main() {  
    Box box1;  
    box1.set(1,2,3);  
    box1.print();  
    box1.set(30,30,30); // box1.height=box1.width=box1.length=30; produces an error  
    box1.print();  
}
```

Box	: (1, 2, 3)
Area	: 22
Volume	: 6
Box	: (30, 30, 30)
Area	: 5400
Volume	: 27000

cannot access directly in main() function

Class with Private Members Example

: Display 6.4 (1/4)

Display 6.4 Class with Private Members (part 1 of 3)

```
1  #include <iostream>
2  #include <cstdlib>
3  using namespace std;

4  class DayOfYear
5  {
6  public:
7      void input( );
8      void output( );
9      void set(int newMonth, int newDay);
10     //Precondition: newMonth and newDay form a possible date.
11     void set(int newMonth);
12     //Precondition: 1 <= newMonth <= 12
13     //Postcondition: The date is set to the first day of the given month.

14     int getMonthNumber( ); //Returns 1 for January, 2 for February, etc.
15     int getDay( );
16 private:
17     int month;
18     int day;
19 };
```

This is an improved version of the class DayOfYear that we gave in Display 6.3.

Private members

Class with Private Members Example

: Display 6.4 (2/4)

```
20  int main( )
21  {
22      DayOfYear today, bachBirthday;
23      cout << "Enter today's date:\n";
24      today.input( );
25      cout << "Today's date is ";
26      today.output( );
27      cout << endl;

28      bachBirthday.set(3, 21);
29      cout << "J. S. Bach's birthday is ";
30      bachBirthday.output( );
31      cout << endl;
32      if ( today.getMonthNumber( ) == bachBirthday.getMonthNumber( ) &&
33          today.getDay( ) == bachBirthday.getDay( ) )
34          cout << "Happy Birthday Johann Sebastian!\n";
35      else
36          cout << "Happy Unbirthday Johann Sebastian!\n";
37
38      return 0;
39  }
```

*Note that the function name
set is overloaded. You can
overload a member function*

Class with Private Members Example

: Display 6.4 (3/4)

```
40 //Uses iostream and cstdlib:
41 void DayOfYear::set(int newMonth, int newDay)
42 {
43     if ((newMonth >= 1) && (newMonth <= 12))
44         month = newMonth;
45     else
46     {
47         cout << "Illegal month value! Program aborted.\n";
48         exit(1);
49     }
50     if ((newDay >= 1) && (newDay <= 31))
51         day = newDay;
52     else
53     {
54         cout << "Illegal day value! Program aborted.\n";
55         exit(1);
56     }
57 }

58 //Uses iostream and cstdlib:
59 void DayOfYear::set(int newMonth)
60 {
61     if ((newMonth >= 1) && (newMonth <= 12))
62         month = newMonth;
63     else
64     {
65         cout << "Illegal month value! Program aborted.\n";
66         exit(1);
67     }
68     day = 1;
69 }
```

just like you can overload any other function.

Mutator function

Mutator function

Class with Private Members Example

: Display 6.4 (4/4)

```
71  int DayOfYear::getMonthNumber( )
72  {
73      return month;
74  }

75  int DayOfYear::getDay( )
76  {
77      return day;
78  }

79  //Uses iostream and cstdlib:
80  void DayOfYear::input( )
81  {
82      cout << "Enter the month as a number: ";
83      cin >> month;
84      cout << "Enter the day of the month: ";
85      cin >> day;
86      if ((month < 1) || (month > 12) || (day < 1) || (day > 31))
87      {
88          cout << "Illegal date! Program aborted.\n";
89          exit(1);
90      }
91  }

92  void DayOfYear::output( )
93      <The rest of the definition of DayOfYear::output is given in Display 6.3.>
```

Accessor functions

Private members may be used in member function definitions (but not elsewhere).

Sample Dialogue

```
Enter today's date:
Enter the month as a number: 3
Enter the day of the month: 21
Today's date is March 21
J. S. Bach's birthday is March 21
Happy Birthday Johann Sebastian!
```

Public and Private Style

- Can place `public` & `private` in any order
 - In fact, `public`, `private`, and then `public` again is possible
 - But, commonly have one `public` and one `private` section
- More typically place `public` first
 - Allows easy viewing of portions that can be USED by programmers using the class
 - Private data is "hidden", so irrelevant to users

Accessor and Mutator Functions

- Object may need to "do something" with its private data
- Call **accessor** member functions
 - Allow object to read data
 - Also called "get member functions"
 - Simple retrieval of member data
- **Mutator** member functions
 - Allow object to change data
 - Also called "set member functions"
 - Manipulated based on application

A Test for Encapsulation

- A different class definition will do the same job
 - The detail on how the data is stored is **NOT** important for a user!

```
class DayOfYear
{
public:
    void input ( );
    void output ( );
```

Exactly the same
accessors and mutators!

```
void set(int newMonth, int newDay);
//Precondition: newMonth and newDay form a possible date.
//Postcondition: The date is reset according to the
//arguments.

void set(int newMonth);
//Precondition: 1 <= newMonth <= 12
//Postcondition: The date is set to first day of the month.

int getMonthNumber ( );
//Returns 1 for January, 2 for February, etc.
int getDay ( );
```

```
private:
```

```
char firstLetter;//of month
char secondLetter;//of month
char thirdLetter;//of month
int day;
```

```
};
```

```
int DayOfYear::getMonthNumber ( )
{
    if (firstLetter == 'J' && secondLetter == 'a'
        && thirdLetter == 'n')
        return 1;
    if (firstLetter == 'F' && secondLetter == 'e'
        && thirdLetter == 'b')
        return 2;
    ...
}
```

Note the different **private** variables!

Thinking Objects

- Focus for programming changes
 - Before → algorithms take the center stage
 - OOP → data is the focus
- Algorithms are still important
 - They simply focus on their data
 - Are "made" to "fit" the data
- Designing software solution
 - Define variety of objects and define how they interact

Structure vs. Class

- In fact, Class in C++ is an extension of Structure in C
- Structures
 - Normally used with all member variables public & no member functions
 - However, in C++, structure can also have public/private variables/functions
 - Still encouraged to use structure as before
- One clear difference
 - For Structure, default is **public**, and for Class, default is **private**

Summary I

- Structure is collection of different types
- Class used to combine data and functions into single unit → object
- Class and structure types can be formal parameters to functions
- Member variables and member functions
 - Can be public → accessed outside class
 - Can be private → accessed only in a member function's definition

Summary 2

- Encapsulation is the key for OOP
 - Separate interface and implementation!
 - Build your own ADT with classes!
 - Using private members can realize the encapsulation

Thank you!

Web: <http://mindlab-snu.github.io>

E-mail: tsmoon@snu.ac.kr